

Implementação e Análise de Desempenho da Simulação de N-Corpos em GPU utilizando CUDA

Bruno A. M. de Abreu Filho¹, Emily I. S. Hamano¹, Pedro Costa Freitas¹

¹Instituto Federal Goiano - Campus Iporá (IF GOIANO) Av. Oeste,
350 – Parque União – Iporá– GO – Brasil

bruno.filho@estudante.ifgoiano.edu.br,

emily.soma@estudante.ifgoiano.edu.br,

pedro.costa2@estudante.ifgoiano.edu.br

Abstract. *This work presents the implementation and performance analysis of the N-body simulation problem using CUDA on GPU architectures. Three versions of the application were developed: a sequential CPU implementation, a parallel GPU implementation using global memory, and an optimized version using shared memory. The experiments evaluated execution time and speedup for different numbers of particles. The results demonstrated significant performance gains for GPU implementations compared to the sequential version, highlighting the efficiency of parallel processing in computationally intensive applications.*

Resumo. *Este trabalho apresenta a implementação e análise de desempenho da simulação do problema de N-corpos utilizando CUDA em arquiteturas GPU. Foram desenvolvidas três versões da aplicação: uma implementação sequencial em CPU, uma implementação paralela em GPU utilizando memória global e uma versão otimizada utilizando memória compartilhada. Os experimentos avaliaram o tempo de execução e o speedup para diferentes quantidades de partículas. Os resultados demonstraram ganhos significativos de desempenho das implementações em GPU em relação à versão sequencial, evidenciando a eficiência do processamento paralelo em aplicações computacionalmente intensivas.*

1. Introdução

O avanço das arquiteturas paralelas permitiu o desenvolvimento de aplicações computacionais capazes de processar grandes volumes de dados em menor tempo. Entre essas arquiteturas, as GPUs (Graphics Processing Units) destacam-se pela elevada capacidade de processamento paralelo, sendo amplamente utilizadas em aplicações científicas, inteligência artificial, simulações físicas e processamento de imagens.

A plataforma CUDA, desenvolvida pela NVIDIA, possibilita o uso de GPUs para computação de propósito geral, permitindo explorar milhares de threads executadas simultaneamente. Dessa forma, aplicações com alto grau de paralelismo podem obter ganhos significativos de desempenho em comparação com implementações sequenciais executadas em CPU.

O problema de N-corpos representa um dos problemas clássicos da computação científica. Nesse problema, múltiplos corpos interagem entre si por meio de forças físicas, como a gravidade, exigindo o cálculo das interações entre todos os pares de partículas do sistema. Como a complexidade computacional cresce proporcionalmente a $O(N^2)$, o custo de processamento torna-se elevado para grandes quantidades de corpos.

Nesse contexto, o objetivo deste trabalho é implementar e analisar o desempenho de uma simulação de N-corpos utilizando CUDA em GPU, comparando os resultados obtidos com uma implementação sequencial em CPU. Além disso, busca-se avaliar o impacto do uso da memória compartilhada (shared memory) na otimização do desempenho da aplicação paralela.

Para isso, foram desenvolvidas três versões da aplicação: uma implementação sequencial em CPU, uma implementação paralela em GPU utilizando memória global e uma versão otimizada utilizando memória compartilhada. Os resultados obtidos foram analisados por meio da comparação dos tempos de execução e do speedup alcançado em diferentes quantidades de partículas.

2. Fundamentação teórica

2.1. Computação Paralela

A computação paralela consiste na execução simultânea de múltiplas tarefas computacionais com o objetivo de reduzir o tempo de processamento e aumentar o desempenho dos sistemas computacionais [Grama et al. 2003]. Diferentemente da computação sequencial, na qual as instruções são executadas uma após a outra, o paralelismo permite dividir um problema em partes menores que podem ser processadas simultaneamente por diferentes unidades de execução.

O crescimento da demanda por aplicações científicas, simulações físicas, inteligência artificial e processamento de grandes volumes de dados impulsionou o desenvolvimento de arquiteturas paralelas. Nesse contexto, as Unidades de Processamento Gráfico (GPUs) tornaram-se amplamente utilizadas devido à sua capacidade de executar milhares de operações simultaneamente.

A computação paralela pode ser aplicada em diferentes níveis, como paralelismo de dados e paralelismo de tarefas. No paralelismo de dados, diferentes partes de um conjunto de dados são processadas simultaneamente utilizando a mesma operação. Já no paralelismo de tarefas, diferentes operações podem ser executadas em paralelo sobre os mesmos dados ou sobre conjuntos distintos.

Além disso, é importante diferenciar paralelismo de concorrência. Enquanto a concorrência refere-se à capacidade de lidar com múltiplas tarefas de maneira organizada, o paralelismo envolve efetivamente a execução simultânea dessas tarefas utilizando múltiplos recursos computacionais.

2.2. Taxonomia de Flynn

A Taxonomia de Flynn, proposta por Michael Flynn em 1966, é uma classificação utilizada para categorizar arquiteturas computacionais com base no número de fluxos de instruções e fluxos de dados processados simultaneamente [Flynn 1966].

Os modelos definidos por Flynn são SISD (Single Instruction, Single Data), SIMD (Single Instruction, Multiple Data), MISD (Multiple Instruction, Single Data) e MIMD (Multiple Instruction, Multiple Data). O modelo SISD representa os computadores tradicionais sequenciais, enquanto o modelo SIMD caracteriza arquiteturas capazes de aplicar a mesma instrução simultaneamente em múltiplos dados, sendo amplamente utilizado em aplicações gráficas e científicas.

O modelo MIMD permite que diferentes processadores executem diferentes instruções sobre diferentes conjuntos de dados simultaneamente, característica presente em sistemas multiprocessados modernos. Já o modelo MISD possui aplicações mais restritas.

As GPUs modernas utilizam conceitos derivados principalmente do modelo SIMD, porém adaptados para o paradigma SIMT (Single Instruction, Multiple Threads), empregado pela plataforma CUDA. Nesse modelo, múltiplas threads executam simultaneamente instruções semelhantes, permitindo elevado grau de paralelismo e melhor aproveitamento dos recursos computacionais.

2.3. Arquitetura CUDA

CUDA (Compute Unified Device Architecture) é uma plataforma de computação paralela desenvolvida pela NVIDIA que permite utilizar GPUs para processamento de propósito geral [NVIDIA 2024]. A plataforma fornece um modelo de programação capaz de explorar o paralelismo massivo presente nas placas gráficas modernas.

Na arquitetura CUDA, as tarefas são organizadas em uma hierarquia composta por threads, blocos e grids. As threads representam as menores unidades de execução e são agrupadas em blocos. Os blocos, por sua vez, formam uma grid responsável pela execução do kernel, que corresponde à função executada na GPU.

As GPUs diferem significativamente das CPUs tradicionais. Enquanto as CPUs são projetadas para baixa latência e execução eficiente de poucas tarefas complexas, as GPUs são otimizadas para alta taxa de transferência de dados e execução simultânea de milhares de threads [Hennessy and Patterson 2017]. Essa característica torna as GPUs especialmente adequadas para aplicações com elevado paralelismo de dados.

Outro conceito importante na arquitetura CUDA é o warp, que consiste em um grupo de threads executadas simultaneamente pela GPU. O desempenho da aplicação pode ser afetado quando ocorre divergência entre threads de um mesmo warp, pois diferentes caminhos de execução reduzem a eficiência do processamento paralelo.

2.4. Hierarquia de Memória em GPUs

O desempenho de aplicações paralelas em GPUs depende diretamente do gerenciamento adequado da memória. A arquitetura CUDA possui diferentes níveis de memória, cada um com características específicas de capacidade, latência e velocidade de acesso [Kirk and Hwu 2016].

A memória global possui maior capacidade de armazenamento, porém apresenta maior latência de acesso. Dessa forma, acessos excessivos à memória global podem reduzir significativamente o desempenho da aplicação.

A memória compartilhada (shared memory) é uma memória de alta velocidade acessível pelas threads pertencentes ao mesmo bloco. Seu uso adequado permite reduzir acessos à memória global, aumentando a eficiência computacional.

Os registradores representam o nível mais rápido da hierarquia de memória, sendo utilizados para armazenar variáveis locais das threads. Entretanto, sua quantidade é limitada, exigindo gerenciamento eficiente dos recursos disponíveis.

Além dessas memórias, a arquitetura CUDA também disponibiliza memória constante e memória de textura, utilizadas em situações específicas para otimização do acesso aos dados.

2.5. Problema de N-Corpos

No problema de N-corpos, o uso adequado da hierarquia de memória é fundamental para reduzir o custo das interações entre partículas, melhorando significativamente o desempenho da simulação.

O problema de N-corpos consiste na simulação das interações entre múltiplos corpos físicos que influenciam uns aos outros por meio de forças, como a gravidade. Esse problema possui ampla aplicação em áreas como astronomia, física computacional, dinâmica molecular e simulações científicas.

Em uma simulação gravitacional, cada corpo exerce força sobre todos os demais corpos do sistema. Dessa forma, o cálculo das interações cresce proporcionalmente ao quadrado do número de partículas, resultando em complexidade computacional $O(N^2)$.

Esse elevado custo computacional torna o problema de N-corpos um candidato ideal para paralelização utilizando GPUs. Como o cálculo das forças sobre cada corpo pode ser realizado independentemente, é possível distribuir as operações entre milhares de threads executadas simultaneamente.

Na implementação paralela utilizando CUDA, normalmente cada thread é responsável pelo cálculo das forças aplicadas a um corpo específico. Essa abordagem permite explorar o paralelismo massivo das GPUs e reduzir significativamente o tempo total de execução da simulação.

Além disso, técnicas de otimização relacionadas ao uso da memória compartilhada e organização eficiente das threads contribuem para melhorar ainda mais o desempenho das aplicações de N-corpos em arquiteturas paralelas.

2.6. Leis de Amdahl e Gustafson

As Leis de Amdahl e Gustafson são utilizadas para avaliar o desempenho e a escalabilidade de sistemas paralelos.

A Lei de Amdahl estabelece que o ganho máximo de desempenho obtido por meio do paralelismo é limitado pela parcela sequencial do programa. Mesmo utilizando grande quantidade de processadores, a parte não paralelizável da aplicação limita o speedup máximo alcançável [Amdahl 1967].

A Lei de Gustafson propõe uma abordagem baseada no aumento do tamanho do problema proporcionalmente ao número de processadores disponíveis. Nesse modelo,

considera-se que sistemas paralelos podem resolver problemas maiores em tempo viável, permitindo melhor aproveitamento do paralelismo [Gustafson 1988].

As duas leis são amplamente utilizadas na análise de desempenho de aplicações executadas em GPUs, permitindo avaliar os limites do paralelismo e a eficiência da escalabilidade computacional.

3. Metodologia

O desenvolvimento deste trabalho foi dividido em etapas envolvendo estudo teórico, implementação da aplicação e análise experimental dos resultados.

Inicialmente, foi realizada uma revisão bibliográfica sobre computação paralela, arquitetura CUDA, hierarquia de memória em GPUs e o problema de N-corpos. Esse estudo forneceu a base necessária para compreender os conceitos envolvidos na paralelização da aplicação.

Posteriormente, foi desenvolvida uma implementação sequencial da simulação de N-corpos utilizando CPU. Essa versão foi utilizada como referência para comparação de desempenho com as implementações paralelas em GPU.

Em seguida, foram implementadas duas versões paralelas utilizando CUDA:

- versão GPU Naive, utilizando apenas memória global;
- versão GPU Shared, utilizando memória compartilhada para reduzir acessos à memória global.

Na simulação, foram geradas posições aleatórias para os corpos, representadas por coordenadas bidimensionais armazenadas em vetores. Cada thread da GPU ficou responsável pelo cálculo das interações de um corpo específico com os demais corpos do sistema.

Os experimentos foram executados considerando diferentes quantidades de partículas ($N = 100, 200$ e 500). Para cada caso, foram medidos os tempos de execução da implementação em CPU e das versões paralelas em GPU.

A avaliação de desempenho foi realizada por meio do cálculo do speedup, definido como a razão entre o tempo de execução da CPU e o tempo de execução da GPU, conforme a Equação 1.

$$Speedup = \frac{T_{CPU}}{T_{GPU}} \quad (1)$$

Os resultados obtidos foram organizados em tabelas e gráficos para facilitar a análise comparativa entre as implementações.

4. Implementação

A aplicação foi desenvolvida utilizando a linguagem Python com suporte à biblioteca Numba CUDA, que permite a execução de kernels paralelos diretamente na GPU.

A implementação foi dividida em três versões principais:

4.1. Implementação Sequencial em CPU

Na versão sequencial, o cálculo das forças entre os corpos foi realizado utilizando laços aninhados. Para cada corpo do sistema, foram calculadas as interações com todos os demais corpos, resultando em complexidade computacional $O(N^2)$.

Essa implementação foi utilizada como base de comparação para avaliar os ganhos de desempenho obtidos pelas versões paralelas em GPU.

4.2. Implementação Paralela GPU Naive

Na versão paralela naive, cada thread da GPU foi responsável pelo cálculo das forças aplicadas sobre um corpo específico. Os dados foram armazenados diretamente na memória global da GPU.

Embora essa abordagem explore o paralelismo massivo da arquitetura CUDA, os acessos frequentes à memória global podem introduzir latência e limitar parcialmente o desempenho da aplicação.

A organização das threads foi realizada em blocos, permitindo distribuir o processamento entre múltiplos multiprocessadores da GPU.

4.3. Implementação Paralela GPU Shared

A versão otimizada utilizando shared memory buscou reduzir a quantidade de acessos à memória global.

Nessa abordagem, parte dos dados necessários para o cálculo das interações foi carregada temporariamente para a memória compartilhada do bloco, permitindo acessos mais rápidos pelas threads pertencentes ao mesmo bloco.

O uso da shared memory reduz a latência de acesso aos dados e melhora a reutilização das informações durante os cálculos das interações entre partículas.

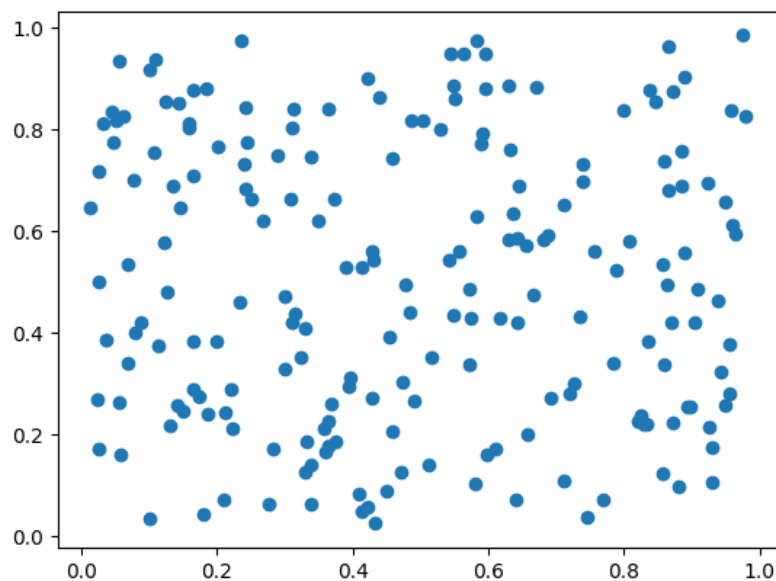


Figura 1. Distribuição das partículas na simulação de N-corpos

4.4. Ferramentas Utilizadas

O desenvolvimento do projeto foi realizado utilizando:

- Python;
- Biblioteca Numba CUDA;
- Google Colab para execução em GPU;
- GitHub para versionamento do código;
- Matplotlib para geração de gráficos.

5. Resultados e Discussão

Tabela 1. Comparação de desempenho entre CPU e implementações em GPU

N	CPU (s)	GPU Naive (s)	Speedup Naive	GPU Shared (s)	Speedup Shared
100	0.279378	0.167180	1.67x	0.223459	1.25x
200	1.520442	0.153264	9.92x	0.220165	6.91x
500	7.690137	0.222632	34.54x	0.225743	34.06x

Os resultados apresentados na Tabela 1 demonstram ganhos significativos de desempenho das implementações em GPU em relação à execução sequencial em CPU.

Para valores menores de N , o ganho de desempenho é relativamente reduzido devido ao custo de transferência de dados entre CPU e GPU e ao overhead de inicialização dos kernels. Entretanto, à medida que a quantidade de corpos aumenta, o paralelismo da GPU passa a ser melhor aproveitado.

Na execução com $N = 500$, a implementação GPU Naive alcançou speedup de aproximadamente 34,54x em relação à CPU, enquanto a versão utilizando shared memory obteve speedup de aproximadamente 34,06x.

Os resultados indicam que ambas as implementações paralelas apresentaram desempenho muito superior à execução sequencial. Entretanto, a otimização utilizando memória compartilhada não apresentou ganho expressivo em relação à versão naive para os tamanhos de entrada utilizados.

Isso pode estar relacionado ao fato de que a quantidade de partículas ainda é relativamente pequena para explorar completamente os benefícios da shared memory. Em problemas maiores, a redução de acessos à memória global tende a produzir ganhos mais significativos.

Além disso, fatores como ocupação da GPU, tamanho dos blocos e custo de sincronização entre threads também influenciam diretamente o desempenho das implementações paralelas.

6. Conclusões e Trabalhos Futuros

Neste trabalho foi realizada a implementação e análise de desempenho da simulação do problema de N-corpos utilizando CUDA em GPU. Foram desenvolvidas três versões da aplicação: uma implementação sequencial em CPU, uma versão paralela em GPU utilizando memória global e uma versão otimizada utilizando memória compartilhada.

Os resultados obtidos demonstraram ganhos significativos de desempenho das implementações paralelas em relação à execução sequencial. O aumento da quantidade de partículas permitiu melhor aproveitamento do paralelismo da GPU, resultando em speedups superiores a 30x para os maiores conjuntos de dados avaliados.

Embora a utilização da memória compartilhada não tenha apresentado ganhos expressivos nos testes realizados, a técnica possui potencial para melhorar o desempenho em simulações com maior quantidade de corpos e maior volume de acessos à memória global.

Dessa forma, o trabalho evidenciou a eficiência do uso de GPUs para aplicações computacionalmente intensivas e demonstrou a importância das técnicas de paralelização e otimização de memória no desenvolvimento de aplicações CUDA.

Como trabalhos futuros, pretende-se ampliar a quantidade de partículas utilizadas nos testes, investigar diferentes configurações de blocos e threads, além de implementar técnicas adicionais de otimização para melhorar ainda mais o desempenho da simulação.

Referências

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30:483–485.
- Flynn, M. J. (1966). Very high-speed computing systems. *Proceedings of the IEEE*, 54(12):1901–1909.
- Grama, A., Gupta, A., Karypis, G., and Kumar, V. (2003). *Introduction to Parallel Computing*. Pearson.
- Gustafson, J. L. (1988). Reevaluating amdahl's law. *Communications of the ACM*, 31(5):532–533.
- Hennessy, J. L. and Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.
- Kirk, D. B. and Hwu, W.-m. W. (2016). *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 3 edition.
- NVIDIA (2024). Cuda c++ programming guide. Disponível em: <https://docs.nvidia.com/cuda/>.